

FreeRTOS 临界段保护与调度器挂起 / 恢复

源码文件：main.c

工程：07-FreeRTOS 临界段代码保护及任务调度器挂起和恢复（正点原子 STM32F4 移植）
生成日期：2026-06-02 · 原文件编码：GBK · 共 275 行

本文档完整收录 main.c 源代码（保留原始行号与注释）。

- **临界段保护**：taskENTER_CRITICAL / taskEXIT_CRITICAL —— 屏蔽中断，连 SysTick 都进不来，Tick 冻结、任务切换完全失效。
- **调度器挂起**：vTaskSuspendAll / xTaskResumeAll —— 只禁止任务切换，SysTick 照常执行、Tick 累计到 xPendedTicks 待恢复后补偿。
- 左侧书签栏可按章节 / 函数跳转；注释以绿色显示，代码以深色显示。
- 注释中原有的两处对照表已改用真实表格呈现（见“两种机制对比” “vTaskDelay() vs delay_ms() 对照表”）。

文件头注释：两种保护机制概述

```
1  /*
2   * 本工程演示 FreeRTOS 两种保护机制的区别：
3   *
4   * 【机制一】临界段保护：taskENTER_CRITICAL / taskEXIT_CRITICAL
5   * - 本质：操作 CPU 的 BASEPRI 寄存器，屏蔽所有可屏蔽中断
6   * - 结果：SysTick_Handler() 进不来，Tick 停止计数，任务切换完全失效
7   * - 内部只能用 delay_ms()（轮询寄存器），不能用 vTaskDelay()（依赖 Tick 中断）
8   * - 演示任务：task1（低优先级进临界段）vs task2（高优先级也无法抢占）
9   *
10  * 【机制二】调度器挂起：vTaskSuspendAll / xTaskResumeAll
11  * - 本质：将全局变量 uxSchedulerSuspended 置位，只禁止任务切换
12  * - 结果：SysTick_Handler() 照常执行，Tick 继续计数，但不做任务切换
13  *   而是把每次 Tick 记入 xPendedTicks，恢复后统一补偿（时序会有轻微漂移）
14  * - 内部同样只能用 delay_ms()，不能用 vTaskDelay()
15  * - 演示任务：task3（低优先级挂起调度器）vs task4（最高优先级也无法运行）
16  *
17  * 【两种机制对比】
```

两种机制对比

	临界段	调度器挂起
SysTick 中断	被屏蔽，不执行	照常执行
Tick 计数	冻结，不累加	继续累加
任务切换	完全失效	禁止
硬件中断	被屏蔽	照常响应
恢复后时间	无偏差	有偏差（补偿 Tick）

```
24  */
25
```

头文件包含与任务配置

```
26 #include "system.h"
27 #include "SysTick.h"
28 #include "led.h"
29 #include "key.h"
```

```

30  #include "usart.h"
31  #include "FreeRTOS.h"
32  #include "task.h"
33
34  /* START_TASK */
35  #define START_TASK_PRIO    1
36  #define START_TASK_STACK_SIZE 128
37  TaskHandle_t start_task_handler;
38  void start_task(void *pvParameters);
39
40  /* TASK1: 演示临界段保护, 优先级2 (低) */
41  #define TASK1_PRIO        2
42  #define TASK1_STACK_SIZE 128
43  TaskHandle_t task1_handler;
44  void task1(void *pvParameters);
45
46  /* TASK2: 临界段期间的观察者, 优先级4 (高) */
47  #define TASK2_PRIO        4
48  #define TASK2_STACK_SIZE 128
49  TaskHandle_t task2_handler;
50  void task2(void *pvParameters);
51
52  /* TASK3: 演示调度器挂起, 优先级3 (低) */
53  #define TASK3_PRIO        3
54  #define TASK3_STACK_SIZE 128
55  TaskHandle_t task3_handler;
56  void task3(void *pvParameters);
57
58  /* TASK4: 调度器挂起期间的观察者, 优先级5 (最高) */
59  #define TASK4_PRIO        5
60  #define TASK4_STACK_SIZE 128
61  TaskHandle_t task4_handler;
62  void task4(void *pvParameters);
63

```

main(): 初始化与启动调度器

```

64  int main(void)
65  {
66      SysTick_Init(168);           /* 初始化 SysTick 硬件定时器, 基于168MHz主频 */
67      NVIC_PriorityGroupConfig(NVIC_PriorityGroup_4); /* 4位抢占优先级, 0位子优先级 */
68      LED_Init();
69      KEY_Init();
70      USART1_Init(115200);
71
72      xTaskCreate((TaskFunction_t)start_task,
73                 (const char* )"start_task",
74                 (uint16_t )START_TASK_STACK_SIZE,
75                 (void* )NULL,
76                 (UBaseType_t )START_TASK_PRIO,
77                 (TaskHandle_t* )&start_task_handler);
78
79      /* 启动调度器: FreeRTOS 接管 SysTick, 将其作为 Tick 中断源。
80       * 此后每次 SysTick 硬件触发 -> SysTick_Handler() -> xPortSysTickHandler()
81       * -> Tick计数+1, 检查阻塞任务是否到期 -> 若需要切换, 挂起 PendSV 中断
82       * -> SysTick_Handler 返回后, PendSV 执行实际的寄存器保存/恢复 (上下文切换) */
83      vTaskStartScheduler();

```

```
84 }
85
```

start_task(): 在临界区中创建任务

```
86 /* 关键点: 在临界段内批量创建任务, 保证4个任务全部就绪后再开始调度,
87 * 避免创建过程中途就被高优先级任务抢占。
88 * taskENTER_CRITICAL 会屏蔽中断, 所以 SysTick 不会触发, 调度器不会切换任务。*/
89 void start_task(void *pvParameters)
90 {
91     taskENTER_CRITICAL();
92
93     xTaskCreate((TaskFunction_t)task1,
94                (const char* )"task1",
95                (uint16_t )TASK1_STACK_SIZE,
96                (void* )NULL,
97                (UBaseType_t )TASK1_PRIO,
98                (TaskHandle_t* )&task1_handler);
99
100    xTaskCreate((TaskFunction_t)task2,
101                (const char* )"task2",
102                (uint16_t )TASK2_STACK_SIZE,
103                (void* )NULL,
104                (UBaseType_t )TASK2_PRIO,
105                (TaskHandle_t* )&task2_handler);
106
107    xTaskCreate((TaskFunction_t)task3,
108                (const char* )"task3",
109                (uint16_t )TASK3_STACK_SIZE,
110                (void* )NULL,
111                (UBaseType_t )TASK3_PRIO,
112                (TaskHandle_t* )&task3_handler);
113
114    xTaskCreate((TaskFunction_t)task4,
115                (const char* )"task4",
116                (uint16_t )TASK4_STACK_SIZE,
117                (void* )NULL,
118                (UBaseType_t )TASK4_PRIO,
119                (TaskHandle_t* )&task4_handler);
120
121    vTaskDelete(start_task_handler);
122    taskEXIT_CRITICAL();
123 }
124
```

临界段保护演示 (task1 / task2)

```
125 /*
126 * 【临界段保护演示】
127 *
128 * task1(prio=2): 每运行5次, 进入临界段延时5秒。
129 * task2(prio=4): 每500ms打印一次, 正常情况下会抢占 task1。
130 *
131 * 临界段期间的现象:
132 * task1 调用 taskENTER_CRITICAL()
133 * -> 屏蔽中断 (操作 BASEPRI 寄存器)
134 * -> SysTick_Handler() 无法进入, Tick 停止计数
135 * -> task2 虽然优先级更高, 但调度器无法切换 (依赖 SysTick/Tick 中断)
```

```

136 * -> task1 独占 CPU 5秒
137 * -> taskEXIT_CRITICAL() 恢复中断，调度器恢复正常
138 *
139 * vTaskDelay 和 delay_ms 的区别：
140 * vTaskDelay(): 阻塞，让任务进入阻塞状态，释放 CPU 给其他任务运行，
141 *             依赖 SysTick 中断 (Tick) 来计时唤醒，临界段内不可用。
142 * delay_ms(): 忙等待，轮询 SysTick->VAL 硬件计数寄存器，
143 *             SysTick 硬件计数器不受中断屏蔽影响，始终在递减，
144 *             临界段内可用，但 CPU 独占，其他任务无法运行。
145 */

```

task1()

```

146 void task1(void *pvParameters)
147 {
148     uint8_t task1_num = 0;
149     while(1)
150     {
151         if (++task1_num == 5)
152         {
153             task1_num = 0;
154             printf("进入临界区! \r\n");
155             taskENTER_CRITICAL(); /* 屏蔽中断: SysTick_Handler 进不来, Tick 停止, 调度器失效 */
156             delay_ms(5000);      /* 忙等待: 轮询 SysTick->VAL 寄存器, 不依赖中断, 临界段内可用 */
157                                 /* 此期间 task2 虽优先级更高, 也无法被调度 */
158             taskEXIT_CRITICAL(); /* 恢复中断, 调度器重新生效 */
159             printf("退出临界区! \r\n");
160         }
161         printf("task1_num:%d\r\n", task1_num);
162         LED1 = !LED1;
163         vTaskDelay(1000); /* 阻塞: 释放 CPU, 等待 1000 个 Tick 后重新就绪 */
164     }
165 }
166
167 /* task2(prio=4): 正常情况下优先级高于 task1, 每500ms抢占一次运行。
168 * 但 task1 进入临界段后, SysTick 中断被屏蔽, vTaskDelay 的计时无法推进,
169 * task2 的阻塞期无法到期, 因此 task2 被彻底阻止, 直到 task1 退出临界段。*/

```

task2()

```

170 void task2(void *pvParameters)
171 {
172     uint32_t task2_num = 0;
173     while(1)
174     {
175         printf("task2_num:%lu\r\n", ++task2_num);
176         vTaskDelay(500);
177     }
178 }
179

```

调度器挂起/恢复演示 (task3 / task4)

```

180 /*
181 * 【调度器挂起/恢复演示】
182 *
183 * task3(prio=3): 每运行5次, 挂起调度器延时5秒。
184 * task4(prio=5): 每500ms打印一次, 正常情况下优先级最高会抢占 task3。
185 *
186 * 调度器挂起期间的现象:

```

```

187 * task3 调用 vTaskSuspendAll()
188 * -> 将全局变量 uxSchedulerSuspended 置位 (计数器+1)
189 * -> SysTick_Handler() 照常触发, xPortSysTickHandler() 照常执行
190 * -> 但发现调度器被挂起, 不做任务切换, 改为将 Tick 记入 xPendedTicks
191 * -> task4 虽然优先级最高, 也无法被切换进来
192 * -> task3 独占 CPU 5秒
193 * xTaskResumeAll() 后:
194 * -> uxSchedulerSuspended 清零, 恢复任务切换
195 * -> 将 xPendedTicks 中积累的 Tick 补偿执行
196 * -> 因此恢复后 task4 的 vTaskDelay 超时时间会出现偏差 (时序漂移)
197 *
198 * 与临界段的关键区别:
199 * 调度器挂起只禁止任务切换, 硬件中断 (包括 SysTick) 照常响应;
200 * 临界段直接屏蔽中断, 连 SysTick 中断本身都进不来。
201 */

```

task3()

```

202 void task3(void *pvParameters)
203 {
204     uint8_t task3_num = 0;
205     while(1)
206     {
207         if (++task3_num == 5)
208         {
209             task3_num = 0;
210             printf("挂起调度器! \r\n");
211             vTaskSuspendAll(); /* 只禁止任务切换, SysTick 中断照常触发, Tick 继续计数 */
212             delay_ms(5000); /* task4 无法被调度, 但硬件中断仍然有效 */
213             xTaskResumeAll(); /* 恢复任务切换, 补偿 xPendedTicks 中积累的 Tick */
214             printf("恢复调度器! \r\n");
215         }
216         printf("task3_num:%d\r\n", task3_num);
217         LED2 = !LED2;
218         vTaskDelay(1000);
219     }
220 }
221
222 /* task4(prio=5): 优先级最高, 正常情况下每500ms运行一次。
223 * task3 挂起调度器后, 虽然 SysTick 中断照常触发,
224 * 但 xPortSysTickHandler 不做任务切换, task4 的阻塞到期信号无法触发切换,
225 * 因此 task4 同样被阻止, 直到 task3 调用 xTaskResumeAll()。*/

```

task4()

```

226 void task4(void *pvParameters)
227 {
228     uint32_t task4_num = 0;
229     while(1)
230     {
231         printf("task4_num:%lu\r\n", ++task4_num);
232         vTaskDelay(500);
233     }
234 }
235

```

总结: vTaskDelay() vs delay_ms()

```

236 /*
237 • vTaskDelay() vs delay_ms() 区别总结

```

238	
239	等待方式不同
240	
241	- vTaskDelay(): 阻塞等待。任务主动放弃 CPU，进入阻塞状态，其他任务可以在此期间运行。
242	- delay_ms(): 忙等待（busy-wait）。CPU 死循环不断读寄存器，什么都不做，其他任务无法运行。
243	
244	依赖的底层机制不同
245	
246	- vTaskDelay(): 依赖 SysTick 中断（Tick）。每次 Tick 中断到来时检查阻塞时间是否到期，到期后把任务移回就绪列表再切换回来。
247	- delay_ms(): 依赖 SysTick 硬件计数器寄存器（SysTick->VAL）轮询。不需要中断，直接读硬件寄存器判断时间。
248	
249	能否在临界段/调度器挂起内使用
250	
251	- vTaskDelay(): 不能用。临界段屏蔽了 Tick 中断，时间无法推进，任务永远无法被唤醒；调度器挂起后也无法完成任务切换。
252	- delay_ms(): 能用。SysTick 硬件计数器不受中断屏蔽影响，始终在递减，轮询读寄存器不依赖中断。
253	
254	对系统的影响
255	
256	- vTaskDelay(): 对系统友好，让出 CPU 给其他任务，是 RTOS 任务中的正确用法。
257	- delay_ms(): 独占 CPU，期间整个系统被"冻住"，只在临界段等特殊场合迫不得已才用。
258	
259	---

vTaskDelay() vs delay_ms() 对照表

	vTaskDelay()	delay_ms()
等待方式	阻塞，释放 CPU	忙等待，独占 CPU
依赖	SysTick 中断（Tick）	SysTick 硬件计数器寄存器
临界段内可用	不能	能
调度器挂起内可用	不能	能
其他任务能否运行	能	不能
RTOS 任务中推荐	是	否（仅特殊场合）
275	*/	